VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification.

Deep Learning and Its Applications (CO3133)

Group 09 – Ryzen9

Instructor: Le Thanh Sach

Hoang Kim Cuong	2352145
Phan Tan Loc	2352712
Tran Manh Thang	2353124

Semester 261 – October 2026

Problem and comparison goal

- One grayscale clothing image is the prediction unit.
- Input: $x \in \mathbb{R}^{1\times 28\times 28}$. Output: one of ten clothing classes.
- Compare Linear, MLP, CNN, GRU, and Transformer under a shared protocol.
- Evaluate predictive quality together with model size and runtime.

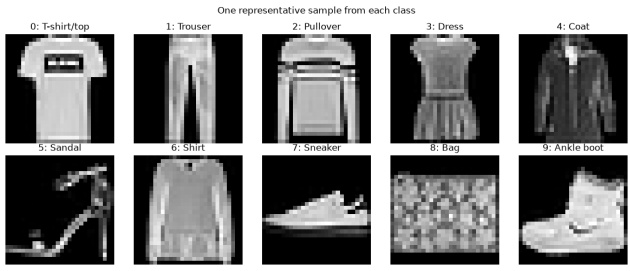
$$z = f_\theta(x) \in \mathbb{R}^{10}, \qquad \hat{y} = \arg\max_{k\in\{0,\dots,9\}} z_k$$

How does image representation affect quality and computational cost?

Fashion-MNIST and exploratory analysis

- 70,000 labeled images.
- 28×28 pixels, one grayscale channel.
- Ten categories.
- 6,000 images per class in the official training set.

No class weighting or resampling.



Balanced counts still leave visually difficult categories.

Official Fashion-MNIST dataset. Original training examples from the EDA.

Shared split and preprocessing

Partition	Images	Per class
Training	54,000	5,400
Validation	6,000	600
Test	10,000	1,000

Stratified split, seed 42.

Shared partitions for all five models.

- Scale pixels to $[0, 1]$.
- Estimate statistics from training images only.
- Keep the original image size and labels.
- No data augmentation.

$$\tilde{x} = \frac{x - 0.286209}{0.353160}$$

Validation selects the checkpoint before held-out test evaluation.

Model input representations

Model	Representation	Shape per image
Linear / MLP	Flattened pixels	784
CNN	Spatial feature maps	$1 \times 28 \times 28$ input
GRU	Sequence of image rows	28 steps, 28 features
Transformer	4×4 patches	49 patch tokens

Representation changes which relationships a model can learn easily.

Linear: from pixels to ten logits

With batch size B, flattening preserves all $1 \cdot 28 \cdot 28 = 784$ pixels:

$$(B, 1, 28, 28) \xrightarrow{\text{Flatten}} (B, 784) \xrightarrow{\text{Linear}(784,10)} (B, 10).$$

For one image, $z = Wv + b$, with $v \in \mathbb{R}^{784}$, $W \in \mathbb{R}^{10\times784}$, $b, z \in \mathbb{R}^{10}$.

- Each class learns 784 pixel weights and one bias.
- Flatten only reshapes the tensor: it has no learned parameters.
- Ten raw logits feed cross-entropy; prediction uses $\arg\max_k z_k$.

$$N_{\text{Linear}} = d_{\text{in}} d_{\text{out}} + d_{\text{out}} = 784 \cdot 10 + 10 = \boxed{7{,}850}.$$

A compact baseline with linear decision boundaries.

MLP: hidden shapes and parameter count

Flatten: $(B, 1, 28, 28) \to (B, 784)$. Each dense layer has $mn + n$ parameters.

Operation	Output shape	Weight / bias shapes	Parameters
Linear $784 \to 256$	$(B, 256)$	256×784 / 256	200,960
ReLU	$(B, 256)$	—	0
Linear $256 \to 128$	$(B, 128)$	128×256 / 128	32,896
ReLU + Dropout 0.2	$(B, 128)$	—	0
Linear $128 \to 10$	$(B, 10)$	10×128 / 10	1,290

$$N_{\text{MLP}} = (784 \cdot 256 + 256) + (256 \cdot 128 + 128) + (128 \cdot 10 + 10) = \boxed{235{,}146}.$$

- ReLU$(a) = \max(0, a)$ adds nonlinearity without changing shape.
- Dropout masks activations during training; disabled at evaluation.
- Dense layers learn pixel combinations without explicit spatial locality.

CNN: feature extraction and spatial shapes

Shapes below omit B and use channels $\times$ height $\times$ width. For unit dilation, each spatial dimension follows

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1.$$

Stage	Configuration	Output shape
Input	Grayscale image	$1 \times 28 \times 28$
Conv1 + BN + ReLU	$1 \to 32$, $k = 3, s = 1, p = 1$	$32 \times 28 \times 28$
MaxPool + Dropout2d	$k = s = 2$; dropout 0.1	$32 \times 14 \times 14$
Conv2 + BN + ReLU	$32 \to 64$, $k = 3, s = 1, p = 1$	$64 \times 14 \times 14$
MaxPool + Dropout2d	$k = s = 2$; dropout 0.2	$64 \times 7 \times 7$

- Conv1: $(28 + 2 - 3)/1 + 1 = 28$; pooling: $(28 - 2)/2 + 1 = 14$.
- Conv2 preserves 14; the second pooling reduces $14 \to 7$.
- BN, ReLU and spatial dropout preserve the tensor shape.

CNN: adaptive pooling and classification head

Operation	Input shape	Output shape
AdaptiveAvgPool2d$(2, 2)$	$(B, 64, 7, 7)$	$(B, 64, 2, 2)$
Flatten	$(B, 64, 2, 2)$	$(B, 256)$
Linear $256 \to 128$	$(B, 256)$	$(B, 128)$
ReLU + Dropout 0.3	$(B, 128)$	$(B, 128)$
Linear $128 \to 10$	$(B, 128)$	$(B, 10)$

- Adaptive pooling averages spatial regions to a fixed 2×2 output.
- Flatten gives $64 \cdot 2 \cdot 2 = 256$ features per image.
- Pooling and Flatten have no trainable parameters.
- The head maps spatial features to ten raw class logits.

Spatial compression keeps the dense head small.

CNN: where do 53,098 parameters come from?

Conv without bias: $C_{\text{out}} C_{\text{in}} k^2$. BN: $2C$ (learned scale γ and shift β).

Layer	Calculation	Trainable parameters
Conv1 (no bias)	$32 \cdot 1 \cdot 3 \cdot 3$	288
BatchNorm2d(32)	$32 + 32$	64
Conv2 (no bias)	$64 \cdot 32 \cdot 3 \cdot 3$	18,432
BatchNorm2d(64)	$64 + 64$	128
Linear $256 \to 128$	$256 \cdot 128 + 128$	32,896
Linear $128 \to 10$	$128 \cdot 10 + 10$	1,290
Total	$288 + 64 + 18{,}432 + 128 + 32{,}896 + 1{,}290$	$\boxed{53{,}098}$

- Convolutions use no bias because BN follows them.
- BN running mean/variance are buffers, not trainable parameters.
- ReLU, pooling, Flatten and dropout contribute zero parameters.

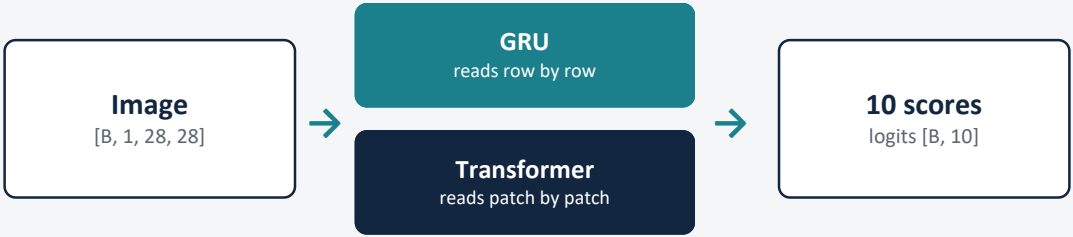
FASHION-MNIST

GRU and Transformer

Two ways to read the same 28×28 image: row by row and patch by patch.

THE TASK

One task, two readers



The highest score is the predicted clothing type.

Each image row is one timestep



28 rows, read top to bottom

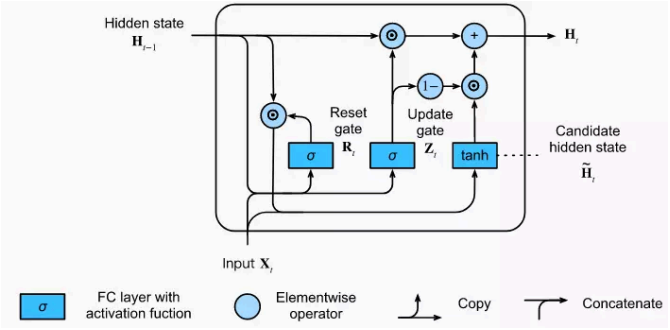
28 **timesteps**
one per row

28 **input size**
pixels in a row

128 **hidden size**
size of the memory

GRU · DIAGRAM

One GRU cell



Reset r
how much old memory to use

Update z
how much old memory to keep

New n
what the current row suggests

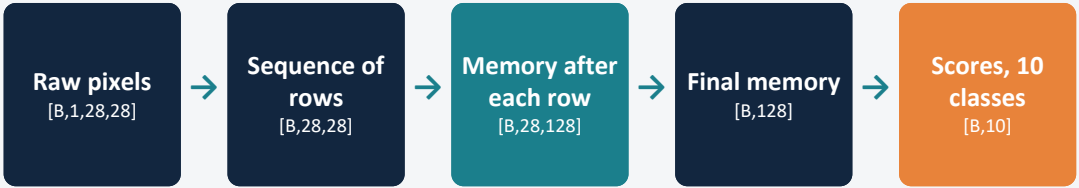
In our notation: R = r, Z = z, candidate $\tilde{H}$ = n.

Formula of each gate

Gate	Formula	Size
Reset r_t	$\sigma(x_t \cdot W_r^T + b_{ir} + h_{t-1} \cdot U_r^T + b_{hr})$	[B, 128]
Update z_t	$\sigma(x_t \cdot W_z^T + b_{iz} + h_{t-1} \cdot U_z^T + b_{hz})$	[B, 128]
New n_t	$\tanh(x_t \cdot W_n^T + b_{in} + r_t \circ (h_{t-1} \cdot U_n^T + b_{hn}))$	[B, 128]
Mix h_t	$(1 - z_t) \circ n_t + z_t \circ h_{t-1}$	[B, 128]

x_t is [B, 28], h_{t-1} is [B, 128], W is [128, 28], U is [128, 128]. $\circ$ means element-wise product.

From pixels to scores



Only h_28, the memory after the last row, goes to the classifier.

GRU · PARAMETERS

Why 61,962 parameters

Parameter	Shape	Calculation	Count
weight_ih	[384, 28]	3 × 128 × 28	10,752
weight_hh	[384, 128]	3 × 128 × 128	49,152
bias_ih	[384]	3 × 128	384
bias_hh	[384]	3 × 128	384
nn.GRU total			**60,672**
Linear 128 → 10	[10, 128]	128 × 10 + 10	1,290
Grand total			**61,962**

```
nn.GRU = 3 × H × (I + H + 2) = 3 × 128 × 158 = 60,672
```

49 patch tokens plus one CLS token



[CLS] is an empty summary sheet that collects information from the whole picture.

Adding a position to each token

The problem

Attention sees a bag of tokens. Shuffle the patches and the CLS result does not change.

The fix

pos_embed [1, 50, 128]: one learned vector per position, shared by all B images.

```
tokens = tokens + pos_embed
```

6,400 parameters

TRANSFORMER · ENCODER

Four stacked encoder blocks

LayerNorm + Attention

+ residual

LayerNorm + MLP

+ residual

× 4 blocks, size stays [B, 50, 128]

Attention

Every token looks at all 50 tokens. 4 heads of 32 dimensions. CLS gathers the summary.

MLP

128 → 512 → 128 with GELU. Each token is processed on its own.

TRANSFORMER · DATA FLOW

From pixels to scores

Stage	Output size	What it means
Input image	`[B, 1, 28, 28]`	grayscale picture
Conv2d 1→128, k=4, s=4	`[B, 128, 7, 7]`	7×7 grid of patches
Flatten + transpose	`[B, 49, 128]`	49 tokens of 128 numbers
Prepend [CLS]	`[B, 50, 128]`	one extra summary token
+ pos_embed, dropout	`[B, 50, 128]`	tokens know their position
4 encoder blocks	`[B, 50, 128]`	tokens exchange information
Final LayerNorm, take [CLS]	`[B, 128]`	one vector for the image
Linear 128 → 10	`[B, 10]`	scores for 10 classes

TRANSFORMER · PARAMETERS

Why 803,338 parameters

Component	Calculation	Count
Patch embedding	4·4·1·128 + 128	2,176
[CLS] token	1 × 128	128
pos_embed	50 × 128	6,400
4 encoder layers	4 × 198,272	793,088
Final LayerNorm	2 × 128	256
Head Linear	128 × 10 + 10	1,290
Total		**803,338**

One encoder layer

Attention: 66,048
MLP: 131,712
2 LayerNorm: 512
Total: 198,272

$$\text{One layer} = 12d^2 + 13d = 196{,}608 + 1{,}664 = 198{,}272 \quad (d = 128)$$

COMPARISON

The two models side by side

	GRU	Transformer
Reads the image as	28 rows in order	49 patches + [CLS]
Gathers information with	Hidden state, last step	Attention, [CLS] token
Knows position by	Order of steps	Learned pos_embed
Parameters	61,962	803,338

Which one predicts better must be tested with experiments.

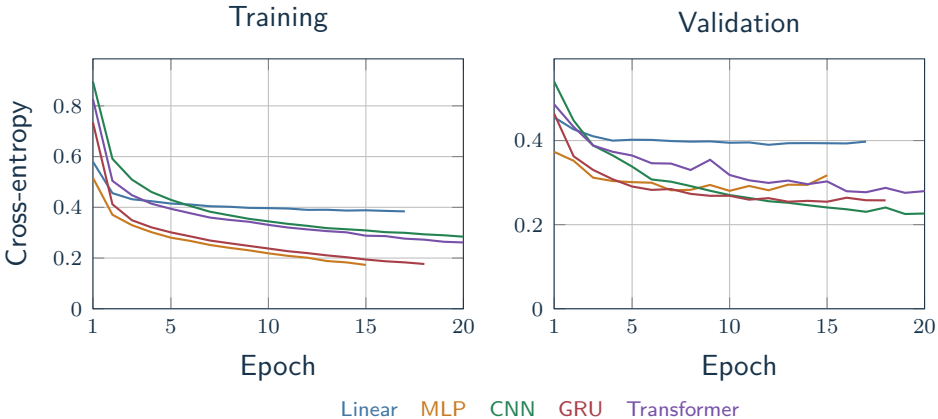
Training and experimental protocol

Loss	Cross-entropy
Optimizer	AdamW
Learning rate	0.001, fixed
Weight decay	0.0001
Batch size	128
Epoch limit	20
Seed	42

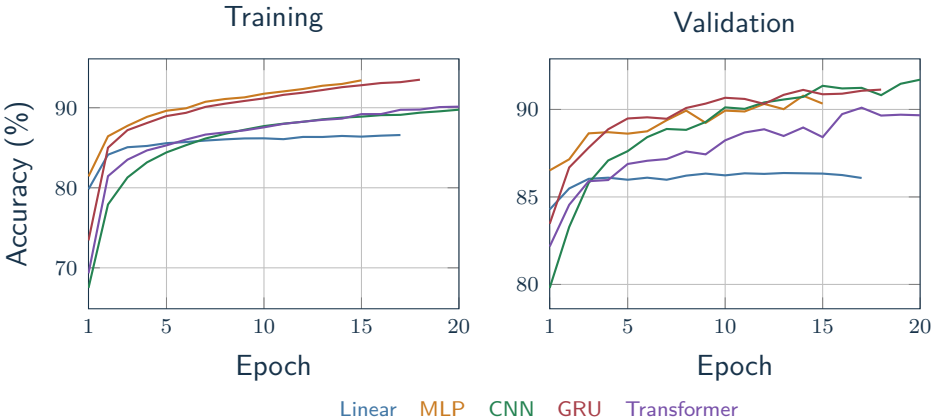
- Early stopping: patience 5, minimum improvement 0.0001.
- Checkpoint: lowest observed validation loss.
- One baseline run per model.
- Shared metrics and test protocol.

NVIDIA GeForce RTX 5070 Ti. FP32 training. No scheduler or mixed precision.

Training and validation loss



Training and validation accuracy



Held-out test performance

Model	Test loss	Accuracy (%)	Macro-F1
Linear	0.4455	84.23	0.8395
MLP	0.3318	88.56	0.8855
CNN	**0.2691**	**90.23**	**0.9023**
GRU	0.2926	89.72	0.8970
Transformer	0.3015	89.16	0.8908

CNN leads these recorded runs on both accuracy and Macro-F1.

10,000 test images. Each model uses its best-validation-loss checkpoint.

Model size and computational cost

Model	Parameters	Epochs	Training (s)	Inference (ms/image)
Linear	7,850	17	290.77	0.0013
MLP	235,146	15	257.38	0.0022
CNN	53,098	20	343.46	0.0070
GRU	61,962	18	305.00	0.0033
Transformer	803,338	20	362.18	0.0232

Linear is smallest and fastest. GRU trades some accuracy for lower inference cost than CNN.

Inference measures batched forward passes. Training includes validation and checkpoint saving.

Upper-body categories remain difficult

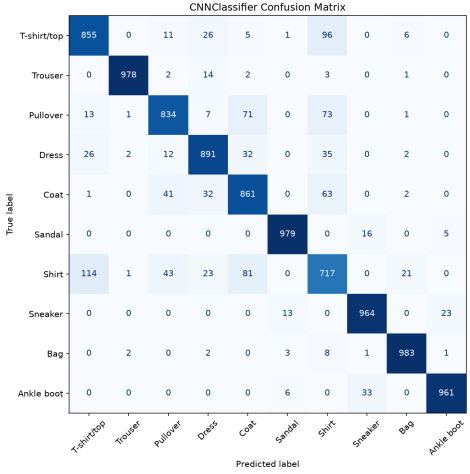
Recall (%)	Linear	MLP	CNN	GRU	Transformer
T-shirt/top	81.8	85.9	85.5	82.9	87.2
Pullover	75.3	75.7	83.4	79.1	88.5
Coat	75.0	87.5	86.1	86.2	81.1
Shirt	50.9	68.7	71.7	71.8	64.8

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The overall winner does not lead every category.

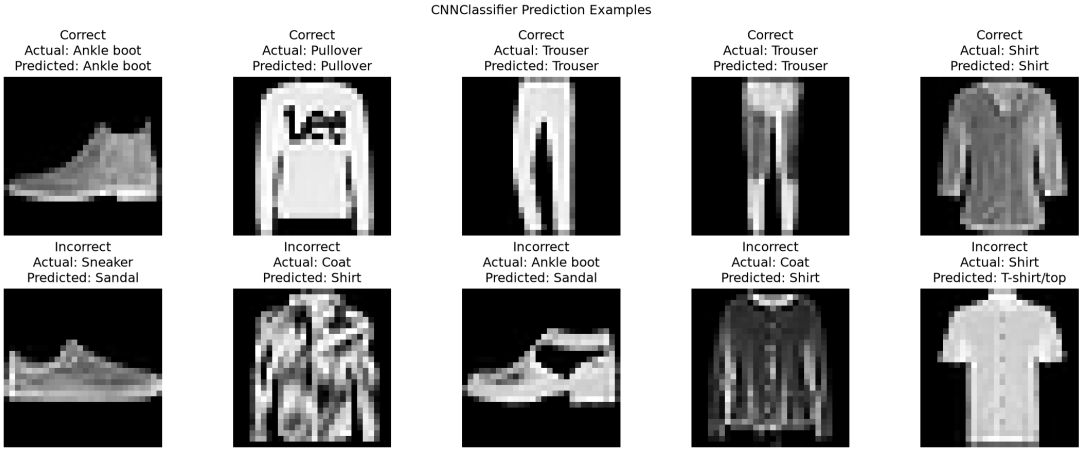
Each test class contains 1,000 images.

CNN confusion matrix



Rows: true class. Columns: predicted class. CNN makes 977 test errors.

Correct and incorrect predictions



First five correct and first five incorrect CNN predictions in test-set order.

Representation and quality-cost trade-offs

- MLP gains 4.33 accuracy points over Linear through nonlinear features, with a much larger parameter count.
- CNN gains 1.67 points over MLP with about 77.4% fewer parameters.
- GRU is 0.51 points below CNN with about 52.9% less inference time.
- Transformer has about 15.1 times CNN's parameters and lower accuracy.

The observed CNN advantage is consistent with useful spatial structure.

Architecture, width, and regularization vary together across these runs.

Limitations and conclusion

- One run per model at seed 42 leaves seed variability unmeasured.
- Shared settings do not individually optimize each architecture.
- Different capacities and epoch counts limit causal comparisons.
- Small grayscale product images limit generalization to retail scenes.

CNN gives the strongest observed predictive result. Linear and GRU offer useful efficiency trade-offs.

Next experiments: repeat seeds, tune on validation data, and test one controlled ablation.

Reproduction and short demonstration

Run from the `assignment1` directory using the released code and configuration.

```
# Train and evaluate a new baseline
python -m experiments.run --model linear --epochs 20 --run-name demo --evaluate-test

# Evaluate the downloaded baseline checkpoint
python -m experiments.run --model linear --evaluate-only --run-id 20260924-124837-513213-m2-
    baseline
```

- Baseline source tag: a1-m2-final-submission-2026-10-17
- Checkpoint archive and recorded run artifacts

AI assistance and sources

- Codex (OpenAI) assisted with slide text, LaTeX structure, and speaker-note drafts.
- Reported metrics originate from executed model runs.
- The group is responsible for final review and explaining the work.

Sources: course handbook Sections 2.3 and 12, the Assignment 1 report, saved baseline logs, and official Fashion-MNIST documentation.